CSCI 5523 Introduction to Data Mining

Spring 2026

Assignment 3 (10 points)

Deadline: April. 1st 11:59 PM CDT

1. Overview of the Assignment

In Assignment 3, you will implement **Min-Hash** and **Locality Sensitive Hashing (LSH)** using PySpark to find similar businesses.

2. Requirements

2.1.Programming Requirements

a. You must use Python to implement all tasks. You can only use standard Python libraries (i.e., external libraries like numpy or pandas are not allowed).
b. **You can only use Spark RDD** (i.e., do not use Spark DataFrame or DataSet)

2.2.Submission Platform

We will use Gradescope to run and grade your submission automatically. We highly recommend that you first test your scripts on your local machine before submitting your solutions.

2.3.Programming Environment

Python 3.9.12 and Spark 3.2.1

You will need to have Java installed to run PySpark. We tested the environment with JDK 11. Please note that PySpark requires Java 8 (but will not work with versions prior to update 8u371), 11, or 17.

We recommend using either Miniconda or Anaconda to create a virtual environment for running your code. To install miniconda or anaconda, you may reference the following links: https://docs.anaconda.com/miniconda/install/ and https://docs.anaconda.com/anaconda/install/

After the conda installation, you can reference the following link to install PySpark: https://spark.apache.org/docs/latest/api/python/getting_started/install.html#using-conda

2.4.Write Your Own Code

Do not use large language models (e.g., ChatGPT, Copilot) or share code with other students!
For this assignment to be an effective learning experience, you must write your own code. We emphasize this point because you will be able to find Python implementations of some of the required functions on the web. Please do not look for or at any such code.
TAs will combine all the code we can find from the web (e.g., Github), other students' code from this and other (previous) sections, and code generated from AI tools for plagiarism detection. **We will report all detected plagiarism to the university.**

3. Yelp Data

For this assignment, we provide three subsets generated from the original review dataset using different filtering conditions, such as restricting reviews to those from California (“**`state`**”==“**`CA`**”). You can access and download the datasets from the “data” folder on Google Drive.

- **`./data/tr1.json`**: 27,424 reviews
- **`./data/tr2.json`**: 952,79 reviews
- **`./data/tr3.json`**: 998,77 reviews

4. Task: Min-Hash + LSH

You need to submit the following files on Gradescope:

a. **Python scripts**: **`task.py`**
   We provide a **script of skeleton code.** Please do not change the content in Figure 1 in your submission. You need to implement the **`main()`** function.

b. [OPTIONAL] You can include other scripts to support your programs (e.g., callable functions).

c. **PDF file** : **`hw3.pdf`** (Described in **Section 4.5**)

```
if __name__ == '__main__':
    start_time = time.time()
    sc_conf = pyspark.SparkConf() \
        .setAppName('hw3') \
        .setMaster('local[*]') \
        .set('spark.driver.memory', '4g') \
        .set('spark.executor.memory', '4g')
    sc = pyspark.SparkContext(conf=sc_conf)
    sc.setLogLevel("OFF")

    parser = argparse.ArgumentParser(description='A3')
    parser.add_argument('--input_file', type=str, default='./data/tr1.json')
    parser.add_argument('--candidate_file', type=str, default='./outputs/candidate.out')
    parser.add_argument('--output_file', type=str, default='./outputs/task.out')
    parser.add_argument('--time_file', type=str, default='./outputs/task.time')
    parser.add_argument('--threshold', type=float, default=0.3)
    parser.add_argument('--seed', type=float, default=0)
    args = parser.parse_args()

    main(args.input_file, args.candidate_file, args.output_file,
        args.threshold, args.seed , sc)
    sc.stop()

    # log time
    with open(args.time_file, 'w') as outfile:
        json.dump({'time': time.time() - start_time}, outfile)
    print('The run time is: ', (time.time() - start_time))
```

Figure 1. Screenshot of skeleton code

4.1.Task Description

In this task, you will implement **Min-Hash, Locality Sensitive Hashing (LSH),** and **Jaccard similarity** to identify similar pairs of users based on ratings in the provided datasets. Please refer to each step for detailed instructions. In particular, for LSH, your goal is to choose the **optimal number of bands and rows.**

Steps :

1. **Create a Binary Rating Matrix (Preprocessing)**

Convert the original rating matrix (Table 1) to a binary rating matrix (Table 2), where each row represents a **user**, and each column a business. Assign a value of **1 if the user has rated the business, and 0 if not**

	Business 1	Business 2	...	Business N
User 1	5			2
User 2		4		4
User 3		3		
User 4	1			4

<Table 1. Original rating matrix>

	Business 1	Business 2	...	Business N
User 1	1	0		1
User 2	0	1		1
User 3	0	1		0
User 4	1	0		1

<Table 2. Binary rating matrix>

2. **Generate Min-Hash Signature**

Define your set of hash functions using a **double hashing scheme** to generate signatures for each business (row) derived from the binary rating matrix. Specifically, each hash function is defined as

$$h_i(x) = (h_1(x) + i * h_2(x)) \bmod M$$

Where each $h_1(x)$ and $h_2(x)$ are defined by

$$h_1(x) = ((a_1 x + b_1) \bmod P), \quad h_2(x) = ((a_2 x + b_2) \bmod P).$$

Here, x is each business index included in the **user** row, $h_i(x)$ is the i-th hash function, i is the hash function index, P is any prime number, and M is the number of bins (usually the number of businesses). You will use **a total of 100 hash functions,** matching those used in **LSH (Step 3)**, to construct the MinHash signature for each user. To construct the base hash functions $h_1(x)$ and $h_2(x)$. You can define any combination of parameters (a_1, b_1, a_2, b_2, P, M) for your implementation.

3. **Apply Locality Sensitive Hashing (LSH)**

Divide each signature into `n_bands` bands, each with `n_rows` rows. If the Min-Hash signatures of two **users** match in at least one band, treat them as candidate pairs. **You should define `n_bands` and `n_rows` directly within your script (without using argparse)** and select **a single (`n_bands` and `n_rows`) pair** that **satisfies the grading criteria for all datasets in Section 4.4** (see Section 3 for data and Section 4.4 for grading).

4. **Calculate Jaccard Similarity**

Compute the **Jaccard Similarity** for candidate pairs based on binary ratings. Please include all business pairs with **Jaccard Similarity >** `threshold`. For example, Table 3 shows the Jaccard Similarity (**user1**, **user2**) = #intersection / #union = 1/4

	Business 1	**Business 2**	**Business 3**	**Business 4**
User 1	0	1	1	0
User 2	1	1	0	1

Table 3. Jaccard Similarity

4.2. Execution Commands

We will run your code using the following command line:

```
python task.py --input_file <input_file> --candidate_file <candidate_file>
--output_file <output_file> --time_file <time_file> --threshold <threshold> --seed
<seed_int>

# Example
# python task.py --input_file tr1.json --candiate_file can1.out --output_file
task1.out --time_file task1.time --threshold 0.3 --seed 0
```

Params:

- **`--input_file`** - the input file (path + file name + extension)
- **`--candidate_file`** - the candidate file (path + file name + extension)
- **`--output_file`** - the output file (path + file name + extension)
- **`--time_file`** - the time file (path + file name + extension)
- **`--threshold`** - threshold for the Jaccard Similarity
- **`--seed`** - random seed for reproducible hash generation

4.3. Output Format

We expect **two output files** (**`candidate_file`** and **`output_file`**) in JSON list (list of dictionaries) format.

1. **Candidate File (`candidate_file`)**

   In step 3, you must write the candidate business pairs to the candidate file using the same tags shown in Figure 2. Each line represents a business pair (i.e., "**`u1`**" and "**`u2`**"). You do not need to generate output for "**`u2`**" and "**`u1`**" since they fundamentally represent the same pairs.

```
{"u1": "mh_-eMZ6K5RLWhZyISBhwA", "u2": "ql0XsKTjM7VeBAUqbphQDw"}
{"u1": "0yoGAe7OKpv6SyGZT5g77Q", "u2": "d6H0HhqyOLiihCG6gbV4cg"}
{"u1": "27km-ZQwZMMq_Yooq3nSIw", "u2": "C76QKXBMh2zDJvCK0Ib6xA"}
{"u1": "27km-ZQwZMMq_Yooq3nSIw", "u2": "Hg5J-w6M0aSVPh9duetNmg"}
```

Figure 2. Expected output format of **`candidate_file`**

2. **Output File (`output_file`)**

   You must write the business pairs with a Jaccard similarity > 0.3 and their similarity in the JSON format using the same tags as shown in Figure 3. Each line represents a business pair (i.e., "**`u1`**" and "**`u2`**"). For each business pair "**`u1`**" and "**`u2`**", you do not need to generate the output for "**`u2`**" and "**`u1`**". You do not need to truncate decimals for the "**`sim`**" values.

```
{"u1": "--4AjktZiHowEIBCMd4CZA", "u2": "7He2tF83YGi8cgbPXByEUg", "sim": 1.0}
{"u1": "--4AjktZiHowEIBCMd4CZA", "u2": "7SgM5DrVSfkaqCpyeKjT0g", "sim": 0.5}
{"u1": "--4AjktZiHowEIBCMd4CZA", "u2": "7ktyPHE-NGnWxarOqjIQiQ", "sim": 0.3333333333333333}
{"u1": "--4AjktZiHowEIBCMd4CZA", "u2": "7wlG7YooB2AUZRyKnqEi1A", "sim": 1.0}
```

Figure 3. Expected output format of **`output_file`**

4.4. Grading (9pt)

Your outputs will be graded by precision and recall, as defined below:

Precision = # true positives / # output pairs

Recall = # true positives / # ground truth pairs

Your goal is to **identify a single (`n_bands`, `n_rows`)** pair that **satisfies the following requirements for all cases.** In order to receive the full score, your precision, recall, and execution time must all meet the specified thresholds under different parameter settings. Each case is worth 3 points, with 1 point assigned to each metric (precision, recall, and execution time). For example, in Case 1, precision must be at least 0.95, recall must be at least 0.9, and the execution time on Gradescope must not exceed 40 seconds.

Case #	Input File	Jaccard Similarity Threshold	Precision	Recall	Time
1	tr1.json	0.3	0.85	0.85	40s
2	tr2.json	0.3	0.90	0.85	50s
3	tr3.json	0.3	0.90	0.90	40s

To facilitate the coding and debugging process, we provide a small dataset **`tr-small.json`**. You can compare your result with the ground truth file (**`tr-small-gt.out`**) by setting the Jaccard similarity threshold, e.g., > 0.3, to find the truly similar pairs and calculate precision and recall accordingly.

4.5. Conceptual Question (1pt)

In the previous task, you implemented Min-Hash and Locality Sensitive Hashing (LSH) to identify similar pairs of businesses using user ratings from the provided dataset.

Question 1: Why is Locality Sensitive Hashing (LSH) used before computing Jaccard similarity? **Please briefly explain your answer** in 2–3 sentences and **submit it to HW3.pdf to the Gradescope.**

5. Grading Criteria

(% penalty = % penalty of possible points you get)

1. You can use your free 5-day extension separately or together. During grading, TAs will count late days **based on the time of submission.** For example, if the due date is 2026/02/24 23:59:59 CDT and your submission is 2026/02/25 05:23:54 CDT, the number of late days would be 1. **TAs will record grace days by default (NO separate Emails)**. However, if you want to save it for next time and treat your assignment as a late submission (with some penalties), **please leave a comment in your submission.**
2. No points will be given if your submission cannot be executed on Gradescope.
3. There is no re-grading. Once the grade is posted on Canvas, we will only re-grade your assignments if there is a grading error. No exceptions.
4. Homework assignments are due at 11:59 pm CT on the due date and should be submitted on Canvas. Late submissions within 24 hours of the due date will receive a 30% penalty. Late submissions after 24 hours of the due date will receive a 50% penalty. Every student has FIVE free late days for homework assignments. You can use these free late days for any reason, separately or together, to avoid the late penalty. There will be no other extensions for any reason. **You cannot use the free late days after the last day of the class.**